Literature Review: Enterprise AI Data Loss Prevention (DLP)

This literature review contextualizes the **SentinelAI** project within the broader academic and industry landscape of Large Language Model (LLM) privacy, prompt interception, and Personally Identifiable Information (PII) detection.

Research Papers & Reviews

Research Paper / Tech Report (Year)	Core Focus	Review & Relevance to SentinelAI
"Are Large Language Models a Threat to Digital Privacy?" *Brown et al. (2022/2023)*	Investigates the capacity of LLMs to memorize and later leak sensitive information (PII, API keys) from their training data or context window.	**Review:** Highlights the exact threat vector SentinelAI mitigates. The paper proves that once data enters an LLM, it cannot be reliably unlearned or contained. **Relevance:** Validates SentinelAI's "interception before transmission" architectural approach as the most secure method for preventing data leaks.
"Evaluating and Mitigating Privacy Risks in Large Language Models" *Mireshghallah et al. (2023)*	Proposes frameworks for assessing how much PII is leaked during user interactions and explores mitigation strategies like differential privacy and input filtering.	**Review:** The study demonstrates that relying on LLM providers for privacy is insufficient. It advocates for client-side or gateway-level data sanitization. **Relevance:** SentinelAI's API Gateway and reverse proxy implementation directly aligns with this paper's recommendation for local, pre-computation data filtering.
"Presidio: Data Protection and Anonymization Services" *Microsoft Research (2020)*	Introduces a customizable, rule-based and ML-driven NLP engine for identifying and anonymizing PII in text.	**Review:** While Presidio is an industry standard for PII detection, it can be heavy and lacks specific tuning for modern LLM developer prompts (like API keys or nested source code). **Relevance:** SentinelAI builds upon similar concepts (Regex Engine + Validation) but extends them with specific modules for Base64/Hex decoding and Source Code Classification, making it uniquely suited for technical prompt scanning.
"Privacy-Preserving In-Context Learning for Large Language Models" *Various / Panda et al. (2023)*	Looks at how contextual prompts can be sanitized without losing the semantic meaning required for the LLM to perform its task effectively.	**Review:** Discusses the trade-off between strict data masking (which breaks LLM utility) and lax masking (which causes leaks). **Relevance:** SentinelAI's Action Engine (BLOCK / WARN / ALLOW) reflects this exact balance, allowing administrators to configure policies that don't arbitrarily break employee workflows while maintaining security.
"A Survey on Privacy in Large Language Models" *Li et al. (2024)*	A comprehensive overview of recent DLP frameworks acts as plugins or proxies to detect and protect PII, credentials, and code in LLM requests.	**Review:** The survey highlights the emerging trend of "Policy-Aware DLP Frameworks" applied to generative AI. **Relevance:** SentinelAI's Audit Log Service and Admin Dashboard map directly to the enterprise requirements outlined here, providing the necessary observability over LLM usage that organizations lack.
"CodeLlama / Codex Privacy Implications & Code Memorization" *Al-Kaswan et al. (2023)*	Analyzes how LLMs memorize proprietary source code and the risks of employees pasting internal IP into public chatbots.	**Review:** Identifying source code is drastically different from identifying standard PII like SSNs due to changing syntax and variables. **Relevance:** SentinelAI's dedicated `code_classifier.py` heuristic is perfectly positioned to address this unhandled edge case, extending protection beyond standard regex matching.
"Extracting Training Data from Large Language Models" *Carlini et al. (2021)*	Performs training data extraction attacks demonstrating that LLMs memorize and regurgitate exact sequences like PII, emails, and UUIDs.	**Review:** A foundational paper proving that once sensitive data enters an LLM, it becomes permanently vulnerable to extraction. **Relevance:** Establishes the absolute necessity of SentinelAI's core premise: intercepting and stripping sensitive data *before* it reaches the LLM.
"Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection" *Greshake et al. (2023)*	Examines how adversaries can remotely exploit LLMs by embedding malicious instructions that cause the LLM to exfiltrate user data.	**Review:** Shows that relying on LLM alignment is not enough to prevent data theft via prompt manipulation. **Relevance:** Validates the need for a dedicated gateway like SentinelAI to monitor and potentially block anomalous outbound prompts that signify exfiltration.
"ProPILE: Probing Privacy Leakage in Large Language Models" *Kim et al. (2023)*	Evaluates how effectively attackers can use carefully crafted prompts to extract PII that was inadvertently included in the LLM's context or training.	**Review:** Highlights the risk of employees pasting PII into chat interfaces. **Relevance:** Directly supports SentinelAI's Browser Extension, which intercepts PII directly in the DOM (ChatGPT/Claude/Gemini interfaces) before the prompt is even submitted.
"How Bad Can It Git? Characterizing Secret Leakage in Public Repositories" *Meli et al. (2019)*	Analyzes the massive scale of accidental API key, token, and database credential leakage by developers into public domains.	**Review:** While focused on Git, the human behavioral flaw (accidental pasting of secrets) translates perfectly to modern AI developer usage. **Relevance:** Validates SentinelAI's dedicated detection modules for API Keys, Tokens, DB Credentials, and its specific encoding-decoder handling.

Conclusion

The architecture of **SentinelAI** is strongly supported by recent literature. Academic consensus clearly indicates that post-generation filtering or relying on LLM vendors for privacy is insufficient. The literature mandates the use of an intervening middleware layer—exactly like SentinelAI's API Gateway and Browser Extension—to execute Regex pattern matching, code classification, and encoding-decoding locally before sensitive organizational data is exposed to external LLMs.